

Family Integrity Guide

Scientific Data Analysis v2.8

Mother, children, and Scientific Workbench under verifiable contracts

Structural and scientific hardening: fail-closed routing, multi-root ownership, strict envelopes, data safety, and domain regressions

English publication edition: September 22, 2026

This is a translation of the preserved private v2.8 guide at commit 001ee0b. Original TeX SHA-256: 1ce624029567acb477df6c286dfcb9e7876c338f238cfd38a3878fffe7847cdc. The original is retained privately under `skills/scientific-data-analysis/.cache/active_references/guides/guia_scientific_data_analysis_v2_8.tex`.

Recorded results below retain their original scope. This translation does not rerun gates, promote the installed family, or establish current release readiness. Synchronization is valid only when the applicable current gates finish without FAIL or ROTO.

Abstract

`scientific-data-analysis` v2.8 turns the v2.7 integration plan into a verifiable release boundary. The family retains its five-root modular architecture while strengthening routing, app-facing contracts, file safety, and scientific capabilities that could previously produce ambiguous or insufficiently traceable results.

Release evidence no longer rests on an isolated rating. The v2.8 gate checks the canonical 61-capability surface, ownership, recursive stub resolution, maintainer isolation, and controlled failures. It is complemented by 118 Python tests per tree, an independent 61-mutation replay, portable smoke, 267 Swift tests, and Scientific Workbench E2E matrices. `plugin-eval` scores remain supporting evidence, not release authority.

Contents

1	Executive overview	1
2	From v2.7 to v2.8	1
3	The family’s structural boundary	2
3.1	Canonical surface	2
3.2	Ownership and execution	2
4	Preserved app-facing contracts	2
5	Scientific and data hardening	2
5.1	Astronomy	2
5.2	Documents and notebooks	3
6	v2.8 regression	3
7	Validation sequence	3
8	Integration with Scientific Workbench	4
9	Interpreting the evidence	4
10	Limitations and deferred work	5
11	Non-goals and prohibited actions	5
12	Conclusion	5

1. Executive overview

v2.8 focuses on family integrity and scientific safety. It does not artificially increase the capability count: it corrects shared invariants, strengthens failure handling, and makes execution from each capability’s owning root verifiable.

The principal decisions are:

- `scientific-data-analysis` remains the mother and compact router.
- `scientific-data-astro`, `scientific-data-documents`, and `scientific-data-notebooks` execute their respective domains.
- `scientific-data-maintainer` retains registries and gates and is not exposed as an ordinary action.
- A `public_surface_registry.yaml` may be a stub; its `canonical_registry` must resolve recursively, with bounds and confinement to the family.
- Every capability has an owning root, which its command uses as `workingDirectory`.
- A broken map or registry must fail in a controlled way; owner selection must not be invented through heuristics.
- Original preservation is mandatory: v2.8-covered paths are tested with SHA hashes and outputs in dedicated workspaces or temporary directories.
- Exact, symlink, hardlink, protected-tree, and output-to-output collisions are blocked before executing the implementation or any output emitter.
- Notebook execution requires human review and explicit `-trust-notebook-code`; confirmation is not presented as an operating-system sandbox.

2. From v2.7 to v2.8

Area	v2.8 change
Current gate	A v2.8 regression verifies all five skills and the complete canonical surface.
Routing	Recursive resolution detects cycles, limits depth, and confines paths to the family.
Ownership	Exact registry/module-map/router parity for 61 capabilities.
Fail-closed behavior	Heuristic fallbacks for known capabilities are removed when canonical metadata is missing.
Common contracts	Status, QA severity, nonfinite values, secret redaction, atomic emitters, and manifests are strengthened and aligned across the five roots.
Astronomy	Risks corrected in MULTISPEC, FITS tables, extraction, RGB, photometric calibration, RV, SB2, and the Li I line.
Documents and notebooks	Output collisions, workspace escapes, nonfinite values, and incomplete validations addressed; code execution remains blocked without explicit trust.
App	Scientific Workbench consumes the installed multi-root family without copying skills into the app.
Guide	This guide replaces v2.7 as the active document while preserving the earlier version.

3. The family's structural boundary

3.1. Canonical surface

The public surface contains exactly 61 unique IDs. Fingerprint and metadata must agree across registry, module map, and router. Stubs resolve to YAML with `entries::`; cycles, excessive depth, or paths outside the five roots produce controlled errors.

3.2. Ownership and execution

Owner	Installed root
mother	<code>~/.codex/skills/scientific-data-analysis</code>
astro	<code>~/.codex/skills/scientific-data-astro</code>
documents	<code>~/.codex/skills/scientific-data-documents</code>
notebooks	<code>~/.codex/skills/scientific-data-notebooks</code>
maintainer	<code>~/.codex/skills/scientific-data-maintainer</code>

`CapabilityCommandBuilder` must construct commands using the owning root's script and use that same root as the working directory. The six maintainer-only IDs stay outside ordinary catalogs and workflows.

4. Preserved app-facing contracts

v2.8 retains compatibility with v1.8, v1.9, and v2.x:

- statuses `PASS`, `WARNING`, `BLOCKED_CONTROLADO`, `FAIL`, and `ROTO`;
- normalized status aliases without hiding the original status;
- canonical artifact types, grouped for readability:
 - `summary_json`, `manifest_json`, `report_md`, and `table_csv`;
 - `preview_png`, `preview_pdf`, `app_preview`, and `fits_visual`;
 - `notebook_ipynb`, `log_txt`, `qa_report`, `handoff_bundle`, and `edited_document`;
 - `unknown` as the explicit fallback.
- strict JSON envelopes without NaN or Infinity;
- `app_hints`, errors, and artifacts compatible with `ToolEnvelopeParser` and `ArtifactDiscovery`;
- a run manifest even when execution ends in a controlled block or failure.

QA severity takes precedence in the app-facing status. A scientific warning must not disappear through later envelope translation. Nonfinite values must be rejected or recorded with an explicit finding.

5. Scientific and data hardening

5.1. Astronomy

- IRAF MULTISPEC types 0 and 1 use the correct linear and logarithmic laws; unsupported type 2 is blocked instead of approximated. The convention is checked against the official [IRAF SPECWCS documentation](#).
- FITS BinTable retains relevant columns and units.
- Spectral extraction is identified as experimental and preserves flux; it is not presented as a complete Horne implementation.

- Exoplanet reduction accepts common `IMAGETYP` aliases, preserves `FILTER`, and scales darks as rates.
- RGB workflows preserve `FILTER`, `EXPTIME`, and `WCS`; quarantine only reverts files owned by its manifest.
- CCD reduction does not apply implicit crops.
- Photometric calibration propagates covariance with absolute sigmas.
- RV and SB2 validate domain, degeneracy, and covariance.
- Li I 6708 windows are disjoint and tested with known equivalent widths.

5.2. Documents and notebooks

- Notebook runners are blocked by default and execute only after receiving `-trust-notebook-code`; declared inputs and outputs remain within the validated boundary.
- Sensitive inputs are not dumped into envelopes or manifests.
- Collisions between inputs, profiles, and Office/LaTeX outputs are detected.
- Semantic comparisons inspect complete tables and text documents.
- Time-series and Dask workflows declare invalid dates and nonfinite values.
- DuckDB and deliverables use strict serialization and unique temporary files.
- An output directory contained in the input is blocked to prevent recursion and re-ingestion of generated outputs.

6. v2.8 regression

The structural gate can run from the mother or maintainer root:

```
python scripts/audit_v2_8_family_integrity_regression.py
```

It checks frontmatter, invocation policies, 61 IDs, fingerprint, metadata parity, scripts by owning root, dynamic cycle/escape rejection, six maintainer-only IDs, symlinks, portability, and failures without tracebacks.

A second independent regression froze a 947-file fingerprint and ran 61 mutations without changing it: 41 public cases (28 astro, five documents, six notebooks, two mother) and 20 maintainer cases through both owner and mother. Every case required return code 2, one strict JSON object on stdout, empty stderr, `BLOCKED_CONTROLADO`, no artifacts or tracebacks, and identical input trees and SHA-256 values before and after.

7. Validation sequence

```
python scripts/sync_public_surface_docs.py --check
python scripts/audit_v2_8_family_integrity_regression.py
python scripts/audit_v2_7_senior_integration_regression.py
python scripts/skill_surface_audit.py \
  --root ../scientific-data-analysis
python scripts/portable_smoke_test.py \
  --profile core --output-dir <temporary-directory>
```

```
# In each of the five roots
python -m unittest discover -s tests -p "test_*.py"
```

```
# In Scientific Workbench
./script/run_swift_tests.sh
```

```
KEEP_E2E_MATRIX=1 ./script/run_e2e_capability_matrix.sh
KEEP_SMOKE_MATRIX=1 ./script/run_smoke_matrix.sh
```

Smoke must cover tables, documents, notebooks, FITS, and optional-backend absence. That absence may result in `BLOCKED_CONTROLADO`: it is not `PASS`, but neither is it contract corruption. Any `FAIL` or `ROTO` prevents a declaration that synchronization is complete.

The app runner builds with SwiftPM, counts the target's `@Test` tests, and fails if fewer than expected execute. This avoids accepting `swift test` that builds without discovering Swift Testing under a particular toolchain.

In the recorded final snapshot, editable and installed families both passed 118/118 Python tests: 14 mother, 31 astro, 36 documents, 27 notebooks, and 10 maintainer. Scientific Workbench passed 267/267 Swift tests, five executed E2E scenarios, and six planning-smoke scenarios. The matrix includes table/CSV, document, notebook, FITS, and mixed packages. A STILTS preflight with a sanitized environment returned the expected nonzero block with `missing_optional_backend`, no artifacts, and no false success.

8. Integration with Scientific Workbench

The app does not contain skill copies. It must:

1. discover all five installed roots;
2. resolve registry stubs recursively;
3. associate every entry with its owning root;
4. execute from that root;
5. hide maintainer from ordinary actions;
6. parse envelopes and discover v1.8/v1.9/v2.x artifacts;
7. keep Jobs, Results, Settings, catalog, and planner consistent with the same metadata;
8. pass Swift tests and representative smoke cases before declaring synchronization.

9. Interpreting the evidence

`plugin-eval` helps identify design and documentation problems, but a 100/A rating with few substantive checks does not establish scientific or runtime correctness. v2.8 authority combines:

1. the current structural gate;
2. unit tests and domain regressions;
3. portable smoke;
4. Scientific Workbench tests and smoke;
5. inspection of artifacts, manifests, and statuses.

Historical gates are retained for traceability. They are not blockers when they only freeze obsolete inventories, paths, or text; any still-applicable invariant must migrate to the v2.8 gate.

With `plugin-eval 0.1.2`, post-hardening reevaluation yields 100/A for mother, documents, and notebooks, and 96/A for astro and maintainer. There are no evaluator failures. All Python lines exceeding 120 characters in the analyzed surface were removed, and maximum aggregate complexity decreased from 55 to 16 in documents, 45 to 13 in notebooks, 20 to 15 in astro, and 20 to 10 in maintainer. All 118 tests were retained: dense modules were split by responsibility and the 123-case

astro matrix became a declarative JSON fixture. Astro deferred cost also fell from 30,059 to 29,429 tokens.

The two 96/A ratings retain one deferred-budget warning each, primarily from suites, contracts, and scientific evidence kept within each root. This is recorded as a real metric limitation; testing and traceability are not removed to force an artificial 100/A.

10. Limitations and deferred work

- Spectral extraction does not yet implement full Horne extraction.
- MULTISPEC type 2 requires an explicit evaluator before it can be enabled.
- FITS cropping with WCS rewriting and exhaustive BLANK handling are outside this release.
- BJD_TDB and other high-precision timing refinements need a dedicated workflow.
- A notebook can write to an external absolute path if its own code instructs it to; the runner does not replace an operating-system sandbox.
- The supported boundary is the public wrappers; directly invoking an internal fixture bypasses that first barrier.
- Tree hardlink scans are bounded at 100,000 entries and fail closed when they cannot demonstrate safety.
- Preflight does not eliminate a TOCTOU race if another process changes the filesystem between check and write.
- v2.8 verifies exact, symlink, hardlink, tree, fixed-output, and output-to-output collisions through domain suites and the 61/61 replay. It does not claim the Cartesian product of all scripts, flags, filesystems, and backends has been enumerated. Scientific Workbench adds a second barrier through `FilesystemSafetyPolicy`.
- Coverage matrices describe measured smoke/regression scope, not exhaustive coverage.

11. Non-goals and prohibited actions

- Do not modify raw data or original scientific files.
- Do not copy skills into the Scientific Workbench app bundle.
- Do not expose maintainer as an ordinary action.
- Do not turn warnings or blocks into false successes.
- Do not hide tracebacks, nonfinite values, or lost warnings.
- Do not declare synchronization with `FAIL` or `ROTO`.
- Do not delete historical guides or contracts.

12. Conclusion

v2.8 turns v2.7's preparatory hardening into a reproducible release boundary: canonical surface, ownership, controlled failures, strict contracts, scientific tests, and multi-root consumption by Scientific Workbench. The version is ready only when structural, domain, portable, and Swift gates agree on a verifiable result without hidden failures.